Artificial Intelligence Concepts and Machine Learning Techniques (AI – 103)

Walaa H. Elashmawi

Professor of AI

walaa.hassan@miuegypt.edu.eg

Academic Profiles: Google Scholar Scopus ResearchGate LinkedIn

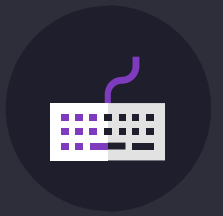
Core Python Programming Foundation

What is Artificial Intelligence (AI) ?

What is AI?

It's the field of creating machines that can **reason, learn, and act** intelligently, often mimicking human cognitive abilities. Think of AI in action:

- **Voice Assistants**
- **Recommendation Systems**
- **Facial Recognition**
- **Self-driving cars**



How do we program it?

AI systems are fundamentally built from the ground up using **code**. To truly understand, build, and innovate in AI, you need to speak its language. We use programming languages like **Python** to:

- **Instruct Machines**
- **Process Data**
- **Design Algorithms**
- **Automate Complex Tasks**

Input Data → Code (Brain) → Intelligent Output

AI is no longer science fiction; it's seamlessly integrated into our daily lives!

What is python?

Python is a high-level, interpreted, general-purpose programming language

Programming languages translate human logic into machine instructions, with different approaches and complexities. Popular examples include:

- **C++**: System software, game development, embedded systems
- **Java**: Enterprise applications, Android apps
- **JavaScript**: Web development (front-end and back-end)
- **Swift**: iOS mobile applications
- **Python**: AI/Data Science, automation, web development

Human Logic

Machine Instructions

Python is a high-level, interpreted, **general-purpose** programming language

Python is exceptionally beginner-friendly and powerful, allowing developers to focus on solving problems rather than managing technical complexity. Its versatility allows it to be used across a vast array of applications:

Web Development - Scientific Computing - **AI/Machine Learning** - Automation & Scripting

What makes it general purpose?

1. **Rich Ecosystem:** The Python Package Index (*PyPI*) hosts thousands of pre-built modules that reduce the need to write code from scratch.

1. **Strong Community Support:** Python boasts one of the largest and most active global developer communities, with extensive documentation, tutorials, and constant library growth.

2. **Platform Independence:** Python runs smoothly on various operating systems (Windows, macOS, Linux) with little to no code changes.

Python is a high-level, **interpreted**, general-purpose programming language

Compilation Vs **Interpretation**

Source code is translated fully into machine code before running *(e.g., C, C++)*	Source code is executed line-by-line at runtime by an interpreter *(e.g., Python, JavaScript)*

Why Python Being Interpreted Is an Advantage

1. **Faster Development**: No compilation step, just write and run
2. **Interactive Development**: Allows for live experimentation
3. **Dynamic Features**: Modify code, types, and objects at runtime
4. **Easier Debugging**: Clear error messages with real-time fixes
5. **Rapid Prototyping**: Ideal for quick tests, data work, and iteration

Python is a **high-level**, interpreted, general-purpose programming language

```cpp
#include <iostream>
#include <vector>
#include <numeric>
#include <iomanip>

int main() {

std::vector<int> numbers = {10, 20, 30, 40, 50};

if (numbers.empty()) {
std::cerr << "Error: Cannot calculate average of empty
vector" << std::endl;
return 1;
}

int sum = std::accumulate(numbers.begin(),
numbers.end(), 0);
double average = static_cast<double>(sum) /
numbers.size();

std::cout << "The average is: " << std::fixed <<
std::setprecision(1) << average << std::endl; return 0;
}
// Output: The average is: 30.0
```

Python abstracts away hardware complexity and memory management, allowing you to write code using human-like syntax and mathematical notation instead of low-level machine instructions.

← **Other Programming Languages (*C++*)** Vs **Python** ↓

```python
numbers = [10, 20, 30, 40, 50]

if not numbers:
    print("Error: Cannot calculate average of
    empty list")

else:
    average = sum(numbers) / len(numbers)

print(f"The average is: {average:.1f}")
# Output: The average is: 30.0
```

Tools used when coding with python

AI Workbench:

A programming environment is the setup where you **write**, **run**, and **test** code. However, setting one up locally can be tricky for beginners

Google Colab is a *cloud-based* environment that needs no setup

Based on **Jupyter** Notebooks, which support:

Code Cells: Run Python code
Text Cells: Add notes using Markdown

Why use google colab?

No Setup Needed

Pre-installed Python
+ AI libraries

Interactive Format

Mix explanations with
live code

Free Hardware Access

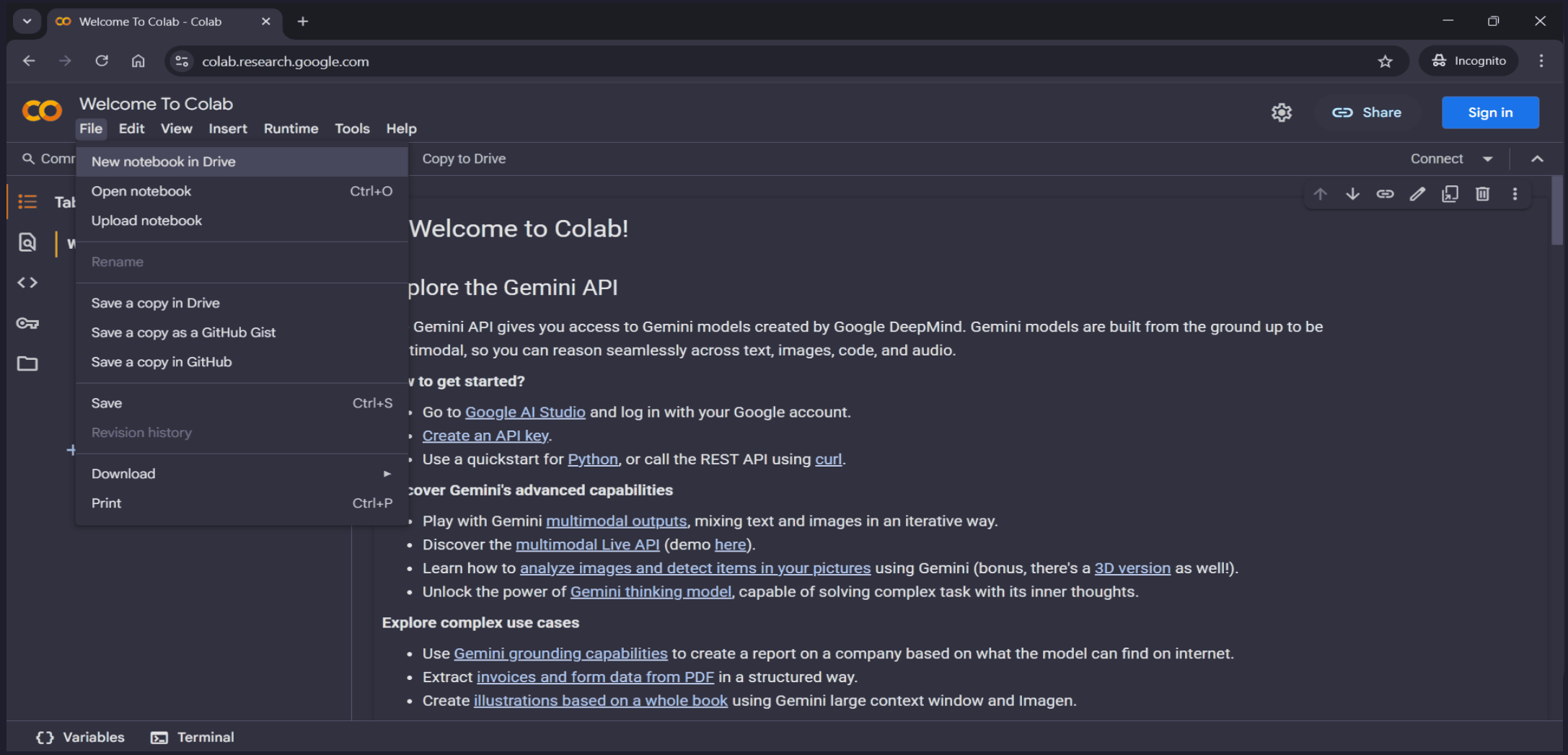
Use Google’s powerful
GPUs & TPUs

Easy Collaboration

Share and collaborate
in real-time

How to get started?

colab.research.google.com



Now let’s continue on over colab

Quick Recap: What did we learn so far?

Variables and Datatypes

how to store numbers, text, and True/False values

Inputs and Outputs

get info from the user and show results with input() and print()

Operators

do math and compare values

Conditional Statements

make decisions with if, elif, and else

Quick Recap: What did we learn so far?

Loops and Loop Control

repeat actions with for and while

Functions and Methods

organize code with reusable blocks that can return values

Lists

store an ordered collection of items to access by position

Dictionaries

store key–value pairs for easy lookups and updates

Now let’s continue on over colab

Quick Recap: What did we learn so far?

So far, we've written our Python code in single blocks or cells. But real-world programs, especially complex AI projects, are rarely just one giant file. They are often split into **multiple, smaller Python files** (called **modules**).

Why is this important for large projects?

1. **Organization:** Keeps code tidy and easy to navigate.
2. **Manageability:** Easier to work on smaller, focused pieces of code.
3. **Collaboration:** Multiple programmers can work on different files simultaneously without conflicts.
4. **Avoiding Overwhelm:** Prevents a single file from becoming too long and complex to understand.

Quick Recap: What did we learn so far?

Functions: They help us reuse blocks of code within *one* program.

Remember this function? It takes a number as its input **parameter** and **returns** the calculated square result

But what if we need this function across multiple projects or modules? Do we need to constantly write it over and over again?

```
# Get square of a number (x^2)

def get_square(number):
        square = number * number
        return square
```

Simple Project Folder Structure

- This file contains the main code that starts and controls how your project runs.

main.py

- Files that store extra functions and helper scripts that make your code cleaner and easier to manage.

tools & utilities/

- This folder holds all the files, like text or images, that your project needs to read or work with.

data/

Simple Project Folder Structure

Project/square.py

```
def get_square(number):
    square = number *
number
    return square
```

This file acts as a module within our project. It's a dedicated place to store a specific, reusable function: `get_square()`.

This is our main program file. Instead of redefining `get_square()`, we use the **import** statement to bring it in from our *square.py* module.

Project/main.py

```
from square import
get_square get_square(4)
# output is 4^2 = 16
```

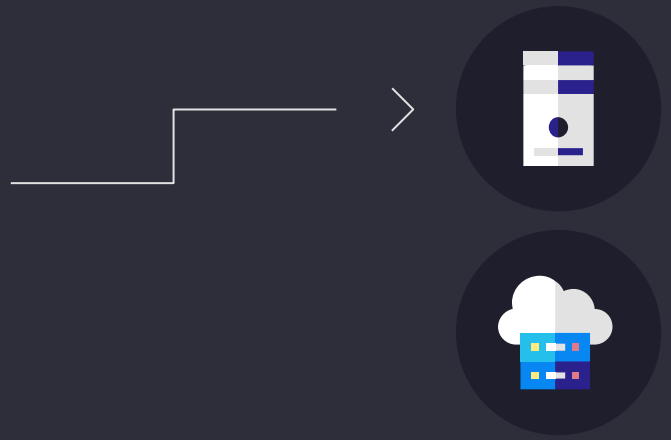
What's an import?

In Python, the import statement is like building a bridge between different parts of your code. It allows you to bring code (functions, variables, classes) from one Python file (called a **module**) into another file where you want to use it.

Imports keep your projects organized, *DRY* (Don't Repeat Yourself), and easy to grow.

Custom Files

reuse your own functions and code saved in other Python files



Libraries

unlock powerful ready-made tools written by others without dealing with their internal implementation

Examples of common libraries

Python Standard Libraries

These are essential collections of code that come **built-in** with Python. You don't need to install them separately; they're ready to import and use right away! They provide common functionalities for everyday programming tasks.

- **math:** For mathematical operations
- **random:** For generating random numbers
- **datetime:** For working with dates and times.
- **os:** For interacting with your computer's operating system (files and folders)

PyPI Third Party Libraries

These are powerful libraries developed by the global Python community. They are not built-in and need to be installed (usually via pip from PyPI, the Python Package Index, like an app store for Python code).

- **NumPy:** For incredibly fast numerical computations and working with arrays
- **Pandas:** For organizing, cleaning, and manipulating tabular data (like spreadsheets).
- **Matplotlib:** For creating professional data visualizations (charts, graphs).

Why do we use libraries?

Libraries contain highly optimized, pre-written code that can replace many lines of manual programming with just one powerful function call. This saves time and reduces errors!

```
scores = [85, 92, 78, ..., 91]
# Imagine a very long list with 1000 scores!

total_sum = 0
count = 0

for score in scores:
    total_sum += score
    count += 1

average = total_sum / count
print(f"Average score is {average}")
# Many lines of code just for average!
```

← **Without libraires** Vs **with libraries** ↓

```
import numpy as np
# Bring in the NumPy toolbox

scores = np.array([85, 92, 78, ..., 91])

average = np.mean(scores)
# One simple line!

print(f"Average score is {average}")
```

Now let’s continue on over colab

الرواد الرقميون
DiGiLiANS

وزارة الاتصالات
وتكنولوجيا المعلومات

THANK you